

TEMA 4

Introducción a la POO. Clases e instancias. Atributos, métodos y mensajes. Encapsulamiento. El modelo de Objetos en C++. Constructores y destructores. Sobrecarga de métodos. Problemas de programación.

Programación Orientada a Objetos

Es un *paradigma* de programación que resuelve problemas mediante conjuntos de objetos que interactúan entre sí.

Objetivos: El software se vuelve fácilmente modificable, escalable y reusable y además se dispone de un modelo natural para representar un dominio.

Ventajas: el software se reutiliza, es más flexible y más cercano a la forma de pensar de las personas.

Introducción a la POO

Objetos

Es una entidad que encapsula datos (*atributos*) que describen el estado del objeto y comportamientos (*métodos*) que son las acciones que el objeto puede realizar dentro de una unidad independiente. Es también una *instancia de una clase*, lo que significa que hereda las características y comportamientos definidos en esta.

Los objetos interactúan entre sí mediante *mensajes*, que es la solicitud para que un objeto ejecute uno de sus *métodos*. Los mensajes pueden incluir *parámetros* que se utilizan por el método para realizar la acción especificada en el mensaje. Los métodos pueden devolver información al objeto emisor del mensaje para que realice otras acciones. Ejemplo: el objeto *Persona* envía el mensaje *comer(pizza)* al objeto *Comida*.

Todo objeto tendrá además un *estado*, que viene determinado por los valores que toman sus datos en un momento determinado y una *identidad*, por la que será siempre diferenciable del resto.

Clases

Es un bloque que contiene los atributos y métodos necesarios para crear un conjunto de objetos según un modelo predefinido. El término atributo se utiliza en análisis y diseño orientado a objetos y el término variable instancia se suele utilizar en programas orientados a objetos.

En C++ los métodos se denominan *funciones miembro* y al momento de su declaración se relacionan con la clase con "::". Ejemplo: `void clase::método (...){...}`

Constructores

Son métodos identificadores de los objetos que llevan el mismo nombre de la clase y se utilizan para inicializar objetos con los mínimos datos necesarios al momento de su creación y así controlar su estado inicial. No poseen valor de retorno y no pueden ser invocados directamente como otro método.

Constructores de copia

Se invoca al crear un objeto a partir de otro, al pasar un objeto por valor a una función o cuando se devuelve un objeto por valor desde una función.

```
Clase(const Clase &copia); <- Forma de declarar un constructor copia  
Clase::Clase (const Clase &c){} <- Otra forma de declararlo
```

Destructor

Método particular para destruir un objeto de una clase determinada liberando su memoria y los recursos que hubiera utilizado (archivos, conexiones a DB, etc.). No reciben parámetros, ni devuelven valores, no pueden ser heredados y deben ser públicos. Es recomendable definirlo en caso de utilizar memoria dinámica o punteros o si se quiere trabajar con logs.

```
Clase::~Clase(){  
    delete[] this->nombre;  
}
```

Siempre que la clase sea padre debe definirse su destructor como *virtual* para evitar que al eliminar una clase hija a través de un puntero de la clase padre no se llame al destructor de la hija y se generen memory leaks o los recursos no se liberen correctamente.

Contención o composición

Es una relación donde una clase *tiene* otras clases como miembros y las clases contenidas se crean y destruyen junto a la principal. Es de propiedad fuerte ya que implica dependencia de existencia: *La clase contenida no tiene sentido por sí sola sin la existencia de la clase contenedora*. Por ejemplo: un motor no tiene sentido sin un auto.

Esta relación implica que la clase contenedora puede utilizar los métodos de la clase contenida, pero si se quiere que la clase contenida acceda a métodos o atributos del contenedor, este debe serle pasado como referencia o declarado como *friend* en la clase contenida.

Pilares de la POO

Abstracción

Es el poder diferenciar entre las propiedades externas y la composición interna de una entidad, ignorando los detalles propios del objeto para poder usarlo como una única unidad comprensible.

Mediante la abstracción se diseñan y fabrican estos sistemas complejos en primer lugar y, posteriormente, los componentes más pequeños de los cuales están compuestos.

En el modelado orientado a objetos significa centrarse en *qué es* y *qué hace* en lugar de *cómo* debe implementarse. Al imaginar los componentes en función de la abstracción (*herramientas abstractas*) es posible centrarse en cómo cada uno interactúa con los otros y no perderse en detalles.

Encapsulamiento

Es el proceso de agrupar datos y operaciones relacionadas bajo la misma unidad de programación, asegurando que la información de un módulo no sea accesible ni visible desde el exterior.

Es el principio fundamental que une **atributos** y **métodos**. Los **atributos** son generalmente **privados**, lo que significa que sólo los **métodos** del objeto pueden acceder a ellos, y los **métodos** son **públicos**, lo que significa que pueden llamarse desde fuera del objeto.

Esto permite controlar el acceso a los datos del objeto y protegerlos de cambios no autorizados.

Es un buen método para reutilizar código, como si fuera herencia, pero la composición debe utilizarse cuando necesitan reutilizarse métodos y no corresponde el uso de herencia. Ej.: la clase *Libre* y la clase *Regular* usan el método *calculadora* (de la clase *Alumno*) para saber si el alumno aprueba o no.

Friend

Permite que una **función o clase externa** tenga acceso a los **miembros privados y protegidos** de otra clase, pero **sin ser parte de ella**.

Herencia

Es un mecanismo que permite crear una clase (**hija**) a partir de una clase existente (**padre**), **heredando sus atributos y métodos**. Al utilizar herencia, puede reutilizarse el código y aplicar *polimorfismo*.

La **clase padre** es general y describe atributos y comportamientos comunes para que la **clase hija** pueda reutilizar todo lo que herede, ampliando sus atributos y comportamiento o redefiniéndolos. Los constructores **no** se heredan, pero pueden invocarse y los métodos y atributos **privados** se heredan conceptualmente, pero no son **accesibles directamente**. Para que haya herencia debe trabajarse con el tipo *protected*.

Un gran problema del uso de herencia es que genera acoplamiento **fijo** (peor que en composición) lo que genera “necesidad”. Por ejemplo, si la función A necesita usar a B, A está acoplada a B. Un peor caso de acoplamiento se da en la **herencia múltiple**, donde una clase hija tiene múltiples padres.

```
class ClasePadre:public ClaseHija; //donde los : indican herencia.
```

Tipos de herencia en C++

- **Public:** Cualquier miembro público de una clase es accesible desde cualquier otra parte donde sea accesible el propio objeto.
- **Protected:** Respecto a las funciones externas, es equivalente al acceso privado, pero con respecto a las clases hijas se comporta como el público.
- **Private:** Sólo son accesibles por los propios miembros de la clase, pero no desde funciones externas o desde funciones de clases hijas. Es el acceso por defecto.

Sobreescritura de métodos

Se da cuando una clase hija **redefine un método** ya existente en la clase padre para cambiar su comportamiento, permitiendo utilizar el mismo nombre del método, pero con diferente comportamiento.

Al momento de sobreescribir métodos es importante resaltar el término **virtual**: habilita el *polimorfismo dinámico*, permitiendo que un método pueda ser sobreescrito en clases hijas y que la

versión “correcta” del método se ejecute **en tiempo de ejecución** y **según el tipo real del objeto** y no del puntero/referencia.

Si no se utilizara virtual se generaría una vinculación estática, donde se resuelve a qué método llamar en **tiempo de compilación** y **según el tipo de puntero/referencia**, no del objeto real. Es por esto que para poder utilizar sobreescritura en la *clase padre* el método debe ser **virtual**. Además, en la *clase hija* el la redefinición debe ser con la misma firma, pero si una hija sobreescrive un método, el método del padre queda inaccesible, es decir, cuando se llame al método **siempre** se tomará el de la hija.

```
virtual void Metodo()
```

Métodos y clases abstract

Una **clase abstracta** es una clase que **no puede instanciarse directamente** porque está diseñada para ser una **base común** para otras clases. Se utiliza para definir un **comportamiento general** pero sin **implementación concreta**. Para que una clase sea abstracta debe tener por lo menos un **método virtual puro**.

Un **método virtual puro** es un método **sin implementación en la clase padre**, lo que **obliga a las clases hijas** a implementarla.

```
virtual void Metodo() = 0; //El = 0 indica que el método es virtual puro
```

En el caso de que una hija no implemente el método, ésta se vuelve abstracta y tampoco puede instanciarse.

Llamada al constructor de la clase padre en una jerarquía de herencia

Es una relación de herencia, cuando se crea un objeto de una hija, **primero se ejecuta el constructor del padre**. Y si el padre tiene un constructor con parámetros, la hija **debe llamarlo explícitamente** en la lista de inicialización. Ejemplo: Alumno es hija de Persona, por lo tanto, al crear un Alumno **se llama primero al constructor de Persona** y luego se ejecuta el constructor de alumno. Esto asegura que el nuevo objeto se construya correctamente en la jerarquía.

Casting

Proceso de convertir un **valor o referencia de un tipo de dato a otro**. En herencia se encuentran dos tipos:

Downcasting

La conversión se hace de la clase base a la clase derivada. Requiere el uso de **casting explícito** y es inseguro si el objeto no es realmente del tipo derivado, generando errores de casteo. Ejemplo: puedo pedir notas a Personas, pero si llega un Profesor se genera un error.

Upcasting

Es la conversión de un objeto derivado a su base. Se realiza de forma **implícita**, por lo que es más seguro. Ejemplo: Profesor y Alumno dan asistencia de manera diferente de forma presencial, pero si se utilizara un formulario dan asistencia de la misma forma.

dynamic_cast

Es un operador de conversión utilizado para convertir punteros o referencias dentro de una jerarquía de clases polimórficas, verificando **en tiempo de ejecución** que la conversión es segura.

Es principalmente utilizado para **downcasting seguro** y solo funciona si la clase base tiene al menos **un método virtual** (o sea, si la **jerarquía es polimórfica**).

Polimorfismo

Es la capacidad de un objeto de **comportarse de diferentes formas según el tipo real**. Ejemplo: el método `cobrar_envio()`; trae el precio de moto o auto según corresponda.

El polimorfismo se implementa con **punteros**, porque debe referenciar a la clase y no ser parte de ella, **virtual**, debido a que habría ligadura estática (static binding) y el compilador determina **en tiempo de compilación** que versión del método usar en base al **tipo** y no del objeto real, y **herencia**, al chequeo por tipo (tipado estático), ya que C++ no sabe si el método admite ese tipo o no.

Si bien es necesario el uso de **virtual**, no es necesario que la clase padre sea abstracta.

Interfaz de un objeto

Es el **conjunto de métodos públicos** que el objeto expone para que otros objetos puedan interactuar con él, **sin saber su implementación interna** permitiendo **encapsulamiento**, facilitando la **modularidad** y haciendo posible el uso de **polimorfismo**.

Se representa mediante una **clase puramente abstracta**.

Otras características

Sobrecarga de operadores

Es **definir su comportamiento para tipos definidos por el programador**, permitiendo usar operadores con objetos como si fueran tipos primitivos.

Los únicos operadores que no pueden sobrecargarse son `::` `.` `.*` y los operadores internos (`sizeof`, `alignof`, etc).

Para `<<` y `>>` debe utilizarse **friend**.

CString

ctype.h

- **int isdigit(int c):** Verifica si un carácter es un dígito.
- **int isalpha(int c):** Comprueba si un carácter es una letra.
- **int isalnum(int c):** Determina si un carácter es una letra o un dígito.
- **int isxdigit(int c):** Revisa si un carácter es un dígito hexadecimal.
- **int islower(int c):** Verifica si un carácter es una letra minúscula.
- **int isupper(int c):** Comprueba si un carácter es una letra mayúscula.
- **int tolower(int c):** Convierte una letra mayúscula a minúscula.
- **int toupper(int c):** Convierte una letra minúscula a mayúscula.
- **int isspace(int c):** Determina si un carácter es un espacio en blanco.
- **int iscntrl(int c):** Revisa si un carácter es de control.
- **int ispunct(int c):** Verifica si un carácter es de puntuación (imprimible, pero no es espacio, ni dígito, ni letra).
- **int isprint(int c):** Comprueba si un carácter es imprimible, incluyendo el espacio.

stdlib.h

- | | | |
|---------------|------------------|--------------------------|
| • atof: float | • atol: long int | • strtol: long |
| • atoi: int | • strtod: double | • strtoul: unsigned long |

stdio.h

- **int getchar(void)**: Lee un carácter desde la entrada estándar.
- **char *gets(char *s)**: Lee una línea de la entrada estándar y la guarda en s.
- **int putchar(int c)**: Escribe un carácter en la salida estándar.
- **int puts(const char *s)**: Escribe la cadena s en la salida estándar, seguida de una nueva línea.
- **int sprintf(char *s, const char *formato, ...)**: Envía una salida formateada a la cadena s.
- **int sscanf(char *s, const char *formato, ...)**: Lee una entrada formateada desde la cadena s

string.h

- **char *strcpy(char *s1, const char *s2)**: Copia la cadena s2 en s1.
- **char *strncpy(char *s1, const char *s2, size_t n)**: Copia n caracteres de la cadena s2 en s1.
- **char *strcat(char *s1, const char *s2)**: Concatena (agrega) la cadena s2 al final de s1.
- **char *strncat(char *s1, const char *s2, size_t n)**: Agrega n caracteres de la cadena s2 al final de s1.
- **int strcmp(const char *s1, const char *s2)**: Compara las cadenas s1 y s2.
- **int strncmp(const char *s1, const char *s2, size_t n)**: Compara los primeros n caracteres de las cadenas s1 y s2.
- **char *strchr(const char *s, int c)**: Busca la primera aparición del carácter c en la cadena s.
- **size_t strcspn(const char *s1, const char *s2)**: Calcula la longitud del segmento inicial de s1 que no contiene caracteres de s2.
- **size_t strspn(const char *s1, const char *s2)**: Calcula la longitud del segmento inicial de s1 que solo contiene caracteres de s2.
- **char *strpbrk(const char *s1, const char *s2)**: Encuentra la primera aparición en s1 de cualquier carácter de s2.
- **char *strrchr(const char *s, int c)**: Localiza la última aparición del carácter c en la cadena s.
- **char *strstr(const char *s1, const char *s2)**: Busca la primera aparición de la subcadena s2 dentro de la cadena s1.
- **char *strtok(char *s1, const char *s2)**: Divide la cadena s1 en "tokens" o fragmentos, usando los caracteres de s2 como delimitadores.
- **void *memcpy(void *s1, const void *s2, size_t n)**: Copia n caracteres desde la ubicación de memoria s2 a s1.
- **void *memmove(void *s1, const void *s2, size_t n)**: Copia n caracteres de s2 a s1; funciona incluso si las áreas de memoria se superponen.
- **int memcmp(const void *s1, const void *s2, size_t n)**: Compara los primeros n caracteres de las áreas de memoria s1 y s2.
- **void *memchr(const void *s, int c, size_t n)**: Busca la primera aparición del carácter c en los primeros n caracteres de s.
- **void *memset(void *s, int c, size_t n)**: Rellena los primeros n caracteres del área de memoria s con el carácter c.
- **char *strerror(int errornum)**: Devuelve un puntero a una cadena que describe el código de error errornum.
- **size_t strlen(const char *s)**: Calcula la longitud de la cadena s.

TEMA 9: STL

Estructuras de Datos y Contenedores

1. Contenedores Secuenciales

a. vector

- `push_back(elemento)` – Añade al final.
- `pop_back()` – Elimina el último elemento.
- `insert(pos, elemento)` – Inserta en posición.
- `erase(pos)` – Elimina elemento en posición.
- `size()` – Número de elementos.
- `capacity()` – Capacidad actual.
- `resize(n)` – Ajusta tamaño.
- `operator[0]` o `at(1)` – Acceso a elementos.
- Declaración:

```
#include <vector>
std::vector<T> v;
```

b. list (lista doblemente enlazada)

- `push_back(elemento)` – Inserta al final.
- `push_front(elemento)` – Inserta al principio.
- `pop_back()` / `pop_front()`.
- `insert(iterador, elemento)`.
- `erase(iterador)`.
- `size()`
- Declaración:

```
#include <list>
std::list<T> lst;
```

c. deque (cola doble)

- `push_back()`
- `push_front()`
- `pop_back()`
- `pop_front()`
- `insert()`
- `erase()`.
- Declaración:

```
#include <deque>
std::deque<T> dq;
```

d. queue (basada en deque)

- `push()` – Inserta al final.
- `pop()` – Elimina del frente.
- `front()` – Elemento en frente.
- `back()` – Último elemento.
- Declaración:

```
#include <queue>
std::queue<T> q;
```

f. stack (LIFO)

- `push()`
- `pop()`
- `top()`
- `size()`.
- Declaración:
- `#include <stack>`

```
std::stack<T> s;
```


2. Contenedores Asociativo

a. map y multimap

Mapeo clave-valor ordenado, con clave única (map) o múltiple (multimap).

- `insert()`: clave, valor
- `find()`, `erase()`
- `operator[]`: mapa[clave] = valor o
int valorDeM = mapa[clave]
- **Declaración:**

```
#include <map>
map<K, V> m;
for (const auto& par : m) {
    cout << par.first << ":"
    << par.second << std::endl;
}
```

```
std::multimap<K, V> mm;
```

c. set y multiset

- **Descripción:** colección ordenada de elementos únicos (set) o múltiple (multiset).
- `insert()`, `find()`, `erase()`.
- **Declaración:**

```
#include <set>
std::set<T> s;
std::multiset<T> ms;
```

Iteradores

Los **iteradores** en C++ son objetos que actúan como punteros abstractos, permitiendo recorrer, acceder y modificar elementos en un contenedor de forma uniforme y segura. Funcionan como enlaces entre los algoritmos y las estructuras de datos, facilitando la manipulación de colecciones como **vector**, **list**, **map**, entre otros.

Funcionamiento y sintaxis básica

Para recorrer un contenedor utilizando un iterador, típicamente se emplea una estructura de ciclo:

```
for (auto it = c.begin(); it != c.end(); ++it) {  
    // Operaciones con *it  
}
```

Tipos de iteradores y sus características

Tipo de iterador	Funciones principales	Operaciones permitidas	Uso típico
iterator	Desde el principio hasta el final del contenedor	Incrementar (++it) y desreferenciar (*it)	Recorrer y modificar elementos en contenedores como vector , list
const_iterator	Igual que iterator , pero sólo lectura (const)	Solo lectura (*it no modificable)	Acceso a datos sin modificar
reverse_iterator	Desde el final hacia el principio (rbegin() a rend())	Decrementar (--it) y desreferenciar (*it)	Recorrer contenedores en orden inverso

Propiedades y métodos principales

Método / Operador	Función	Ejemplo de uso
<code>operator*()</code>	Desreferenciar: obtener referencia al elemento	<code>*it</code>
<code>operator++()</code>	Avanzar al próximo elemento	<code>++it</code>
<code>operator--()</code>	Retroceder al elemento anterior (en iteradores bidireccionales)	<code>--it</code>
<code>operator==()</code> y <code>operator!=()</code>	Comparar dos iteradores	<code>it == c.end(), it != c.begin()</code>
<code>-></code>	Acceder a miembros de objetos apuntados	<code>it->miembro</code>

Operaciones adicionales útiles

- `c.begin()`, `c.end()` — inicio y fin del recorrido.
- `c.rbegin()`, `c.rend()` — recorrido en orden inverso.
- `distance(it1, it2)` — calcular distancia entre iteradores.
- `advance(it, n)` — avanzar o retroceder un iterador `n` pasos.

Ejemplos prácticos

Recorrer un vector e imprimir elementos

```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    for (auto it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << std::endl;
}
```

Recorrer en orden inverso usando **reverse_iterator**

```
#include <deque>
#include <iostream>

int main() {
    std::deque<int> d = {1, 2, 3, 4, 5};
    for (auto rit = d.rbegin(); rit != d.rend(); ++rit) {
        std::cout << *rit << " ";
    }
    std::cout << std::endl;
}
```

Uso con **const_iterator** para acceso en modo lectura

```
for (auto it = v.cbegin(); it != v.cend(); ++it) {
    std::cout << *it << " ";
}
```

Algoritmos

- `copy(inicioOrigen, finOrigen, inicioDestino)`: Copia un rango de elementos desde una fuente a un destino.
- `find(inicio, fin, const T& valor)`: Un iterador al primer elemento que coincide con valor. Si no lo encuentra, devuelve el iterador fin.
- `remove(inicio, fin, const T& valor)`: "Elimina" elementos de un rango que coinciden con un valor. No reduce el tamaño del contenedor. Mueve todos los elementos que *no* coinciden con el valor al principio del rango y devuelve un iterador al "nuevo" final lógico.
- `sort(inicio, fin)`: Orden ascendente
- `sort(inicio, fin, comparador)`:
- `reverse(inicio, fin)`: Invierte el orden de los elementos en un rango.
- `min_element(inicio, fin)`: Encuentra el elemento más pequeño en un rango. Devuelve un iterador al elemento más pequeño.
- `max_element(inicio, fin)`: Encuentra el elemento más grande en un rango. Devuelve un iterador al elemento más grande.
- `count(inicio, fin, const T& valor)`: Cuenta la cantidad de veces que aparece un valor específico en un rango. Devuelve un nro con el total.
- `accumulate(inicio, fin, T total)`: Suma total con todos los elementos.
- `for_each(inicio, fin, funcion)`: Aplica una función (o lambda) a cada elemento de un rango.
- `transform(inicio1, fin1, inicioDestino, operacion)`: Aplica operación a cada elemento de inicio1 a fin1 y lo almacena a partir de inicioDestino.

Manejo de Archivos

1. Archivos de Texto

Apertura para lectura:

```
#include <fstream>
#include <string>

// Abrir para lectura
ifstream fin("archivo.txt");
while (fin.is_open()) {
    string linea;
    while (fin.read((char *) &linea, sizeof(string)) {
        // Procesar línea
    }
    fin.close();
}
```

Apertura para escritura:

```
// Abrir para escritura
ofstream fout("salida.txt");
if (fout.is_open()) {
    fout << "Hola mundo" << endl;
    fout.close();
}
```

Modos:

```
//Abrir en modo truncamiento (borra el contenido previo)
ofstream fout("archivo.txt", ios::trunc);
```

*Nota: **trunc** es la opción por defecto al abrir para escritura en **ofstream**, así que generalmente no es necesario especificarlo explícitamente, pero puede hacerse para mayor claridad.*

```
ofstream outFile("archivo.txt", ios::app); //Modo append
```

```
// También en modo lectura y escritura:
fstream file("nombre.txt", ios::in | ios::out);
```

```
//Entrada y salida con trunc
fstream file("archivo.txt", ios::in | ios::out | ios::trunc);
```

```
//Entrada y salida con append  
fstream file("archivo.txt", ios::in | ios::out | ios::app);
```

Uso con iteradores:

```
#include <iterator>  
#include <algorithm>  
#include <vector>  
  
vector<int> v;  
copy(istream_iterator<int>(fin),  
istream_iterator<int>(),  
back_inserter(v));
```

2. Archivos Binarios

Apertura:

```
#include <fstream>  
  
ofstream binOut("archivo.bin", ios::binary);    // Escritura binaria  
ifstream binIn("archivo.bin", ios::binary);    // Lectura binaria  
  
if (binOut.is_open() && binIn.is_open()) {  
    // Leer o escribir datos binarios  
}
```

Ejemplo para guardar un objeto:

```
struct Persona {  
    char nombre[50];  
    int edad;  
};  
  
Persona p;  
strcpy(p.nombre, "Juan Perez");  
p.edad = 30;  
  
ofstream archivo("personas.bin", ios::binary);  
if (archivo.is_open()) {  
    archivo.write((char*)&p, sizeof(Persona));  
    archivo.close();  
}
```

Para leer:

```
Persona p;
```

```
ifstream archivo("personas.bin", ios::binary);
if (archivo.is_open()) {
    archivo.read((char*)&p, sizeof(Persona));
    archivo.close();
    cout << "Nombre: " << p.nombre;
    cout << "Edad: " << p.edad ;
}
```

Resumen de las opciones más comunes:

- `ios::in` — abrir para leer.
- `ios::out` — abrir para escribir.
- `ios::app` — abrir para agregar al final.
- `ios::trunc` — truncar (borrar contenido previo).
- `ios::binary` — modo binario.

Ejemplo archivos: